

SA-MIRI Course Schedule (03/12 Tentative)

day	Wednesday	day	Friday
03/12	Optimization and Scaling of Large Language Models Presentation Pg	05/12	Pk/Ph
10/12	Pk Presentation Ph Presentation Pi	12/12	Pk
17/12	Presentation Pl (Looking Forward: HPC and AI Futures) All groups will present	19/12	Pk tentative “attendance not required (for students returning home)”

SA-MIRI Course Schedule (03/12 Tentative)

Practical Session	acronym	Book Chapter included
Pk	OPT & SCAL LLM	task 15.1 – Baseline experiment using facebook/opt-1.3b task 15.2 – Finding the out-of-memory limit task 15.3 – Mixed precision training task 15.4 – Model precision task 15.5 – Increasing batch size with model precision task 15.6 – Enabling flash attention task 15.7 – Increasing batch size with flash attention task 15.8 – Using the liger kernel task 15.9 – Augmenting batch size due to liger kernels task 15.10 – Scaling on multiple gpus task 15.11 – Final reflections
Pl	FUTURE	task 16.1 – Mapping the pendulum shifts of the triad task 16.2 – Exploring the new giants of compute task 16.3 – Powering the future of ai task 16.4 – Charting the role of quantum in future supercomputing task 16.5 – From data farms to ai preserves task 16.6 – Embodiment as a source of learning task 16.7 – The European AI Act and the Quest for Computational Sovereignty task 16.8 – Bringing Inference to the Edge

PI assignment

- <https://docs.google.com/spreadsheets/d/1agz0oC38iW7zIOM9S120pphnR7PSw9X228Z-3cup9L4/edit?usp=sharing>

PI - FUTURE						GROUP A	GROUP B	GROUP B
TASK 16.1 — MAPPING THE PENDULUM SHIFTS OF THE TRIAD						1	12	
TASK 16.2 — EXPLORING THE NEW GIANTS OF COMPUTE						5	2	
TASK 16.3 — POWERING THE FUTURE OF AI						4	8	13
TASK 16.4 — CHARTING THE ROLE OF QUANTUM IN FUTURE SUPERCOMPUTING						9	3	
TASK 16.5 — FROM DATA FARMS TO AI PRESERVES								
TASK 16.6 — EMBODIMENT AS A SOURCE OF LEARNING						11		
TASK 16.7 — The European AI Act and the Quest for Computational Sovereignty								
TASK 16.8 — Bringing Inference to the Edge						6	7	

	name	GROUP	
1	ALONSO GARCIA, MARTÍ;	4	
2	ANDREU GERIQUE, DAVID;	4	
3	ARNABOLDI, LEONARDO;	2	
4	ARÓSTEGUI GARCÍA, ALBERTO;	5	
5	AUBERTIER, THOMAS;	9	
6	CRAMER, TIM;	8	
7	DAHLBOM, IRMA;	8	
8	DÍAZ CALDERÓN, MARC;	3	
9	ESCOLÀ CURCÓ, POL;	13	
10	HENLE, PATRICK;	11	
11	HÖPFL, LEON;	7	
12	KARG, CHISTIAN;	12	
13	KOCH ENES, ANTONIO MANUEL;	11	
14	KOLEVA, ELIZABET;	2	
15	LÓPEZ CASTILLÓN, PAU;	3	
16	MENA DOZ, VÍCTOR;	5	
17	MOLDOVEANU, ANDREI;	6	
18	OLIVERA LOYOLA, JUAN JOSE;	13	
19	PAXTON, LAURA;	10	
20	RABADAN ARROYO, RAÜL;	1	
21	RIOS LÓPEZ, ADRIÁN;	1	
22	RITTER, MAXIMILIAN FALK;	10	
23	SELIGA, JAKUB;	9	
24	SIMON, WILLIAM;	7	
25	SOLDERA GIRELLI, VALÉRIA;	6	
26	WALSH, ERIC;	12	

PI: Looking Forward

- **Tasks included:**

- Assigned task

- **Deliverable & Evaluation:**

- Each group must upload a single PDF to the *racó@FIB* intranet.
- You may include as many slides as you consider necessary.
- The expected presentation length is **3 to 8 minutes**.

- **In class (evaluation day):**

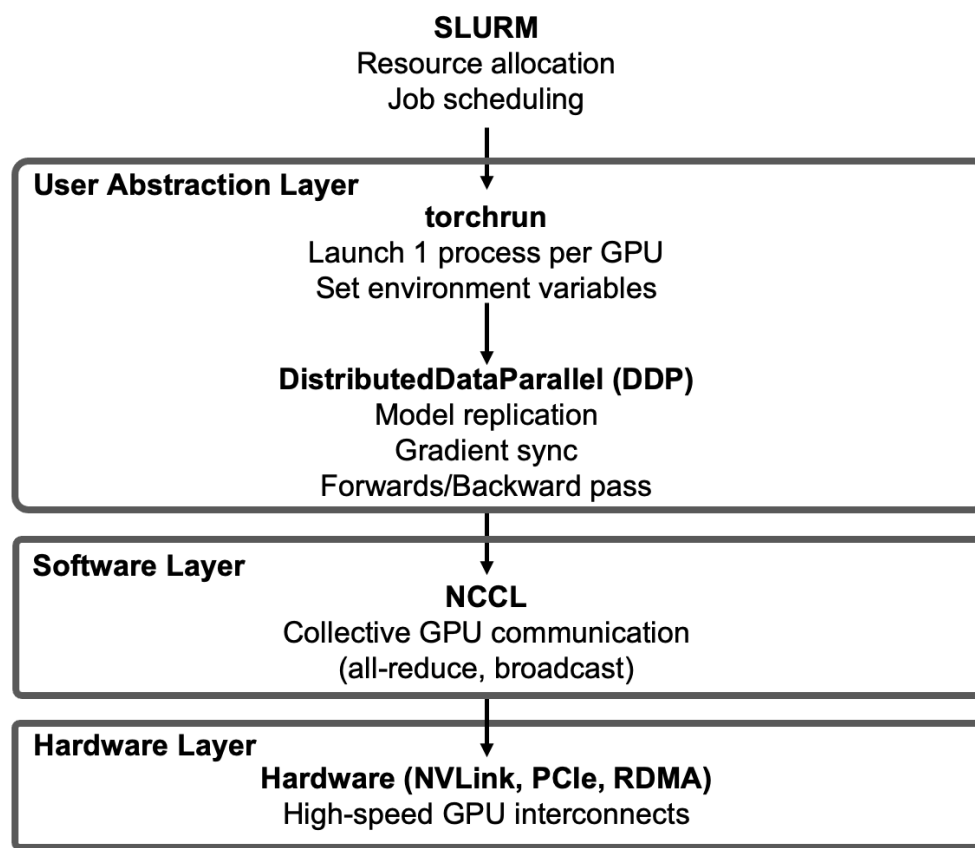
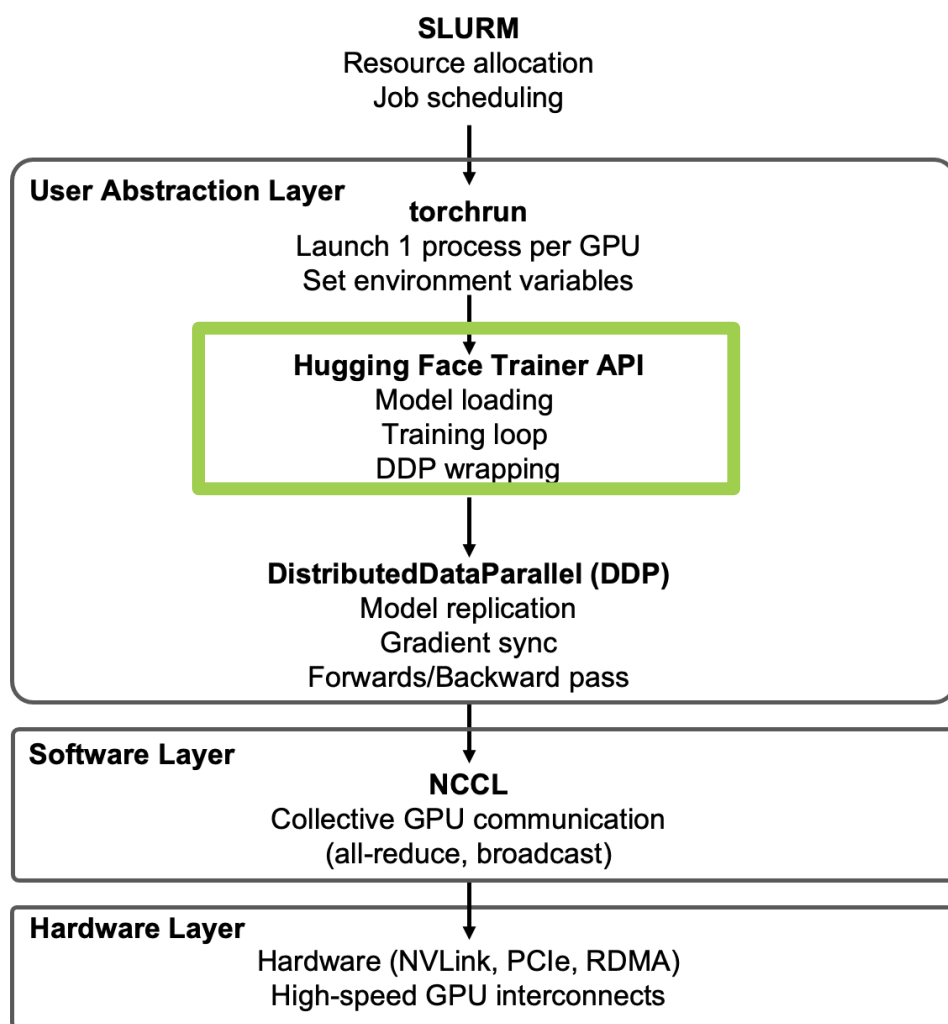
- All groups will present in class. Presentations must be clear, concise, and straight to the point.



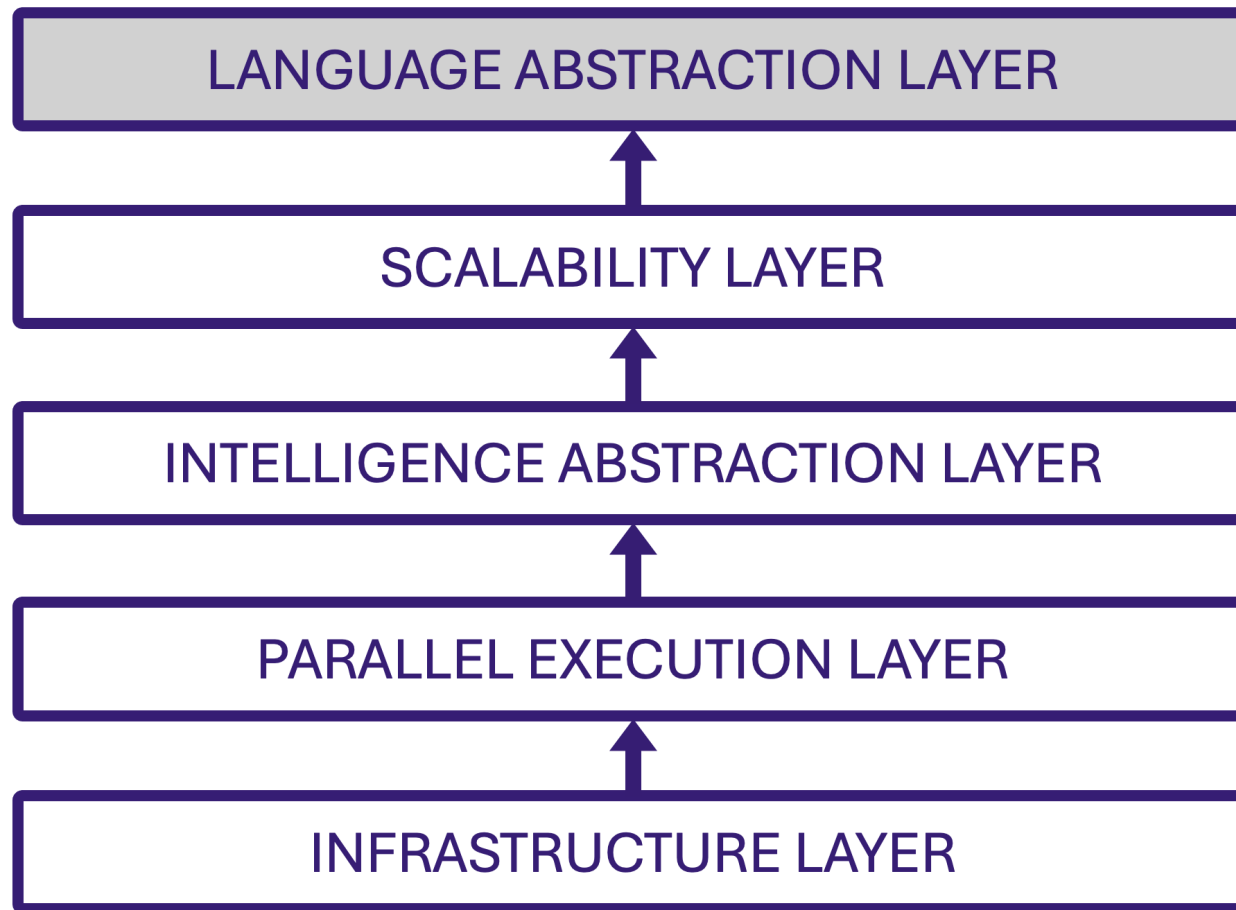
What will we do?

Large LLMs at Scale

- The complexities of distributed LLM training can be managed **using the Hugging Face Trainer API**, which provides a high-level abstraction over PyTorch DDP and NCCL.



PART V of the HPC4AI book



PART V overview

- **Chapter 13 – Introduction to LLMs**

- Provides the essential theoretical and conceptual foundation for LLMs and introduces Hugging Face as the de facto standard for LLM work.

- **Chapter 14 – End-to-End LLM Workflow**

- LLM lifecycle from model access to execution on the supercomputer (training+ inference)

- **Chapter 15 – Optimization and Scaling of LLMs**

- Applying systematic optimization and parallelization to achieve maximum efficiency and scalability for LLM training.

MAIN CHAPTER

Lab 12 Core Objective

■ To efficiently train LLMs by

1. **maximizing performance per GPU** and

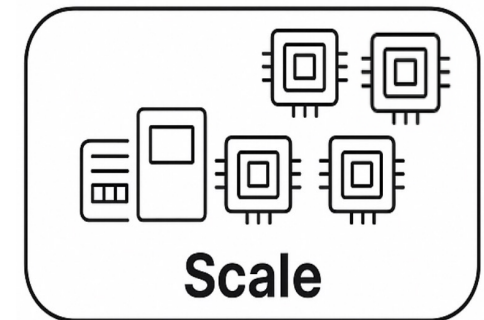
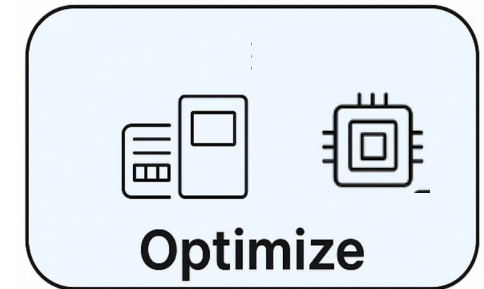
Optimize the training of an LLM—specifically the facebook/opt-1.3b model—on a single GPU H100 to understand the main performance bottlenecks.

2. **scaling effectively across many devices**

Once the single-GPU configuration is efficient, apply the same setup to distributed training and analyze how performance scales up to 32 GPUs.

– **Methodology:**

- Apply systematic, hands-on optimization techniques and parallelization strategies using a real-world case study.



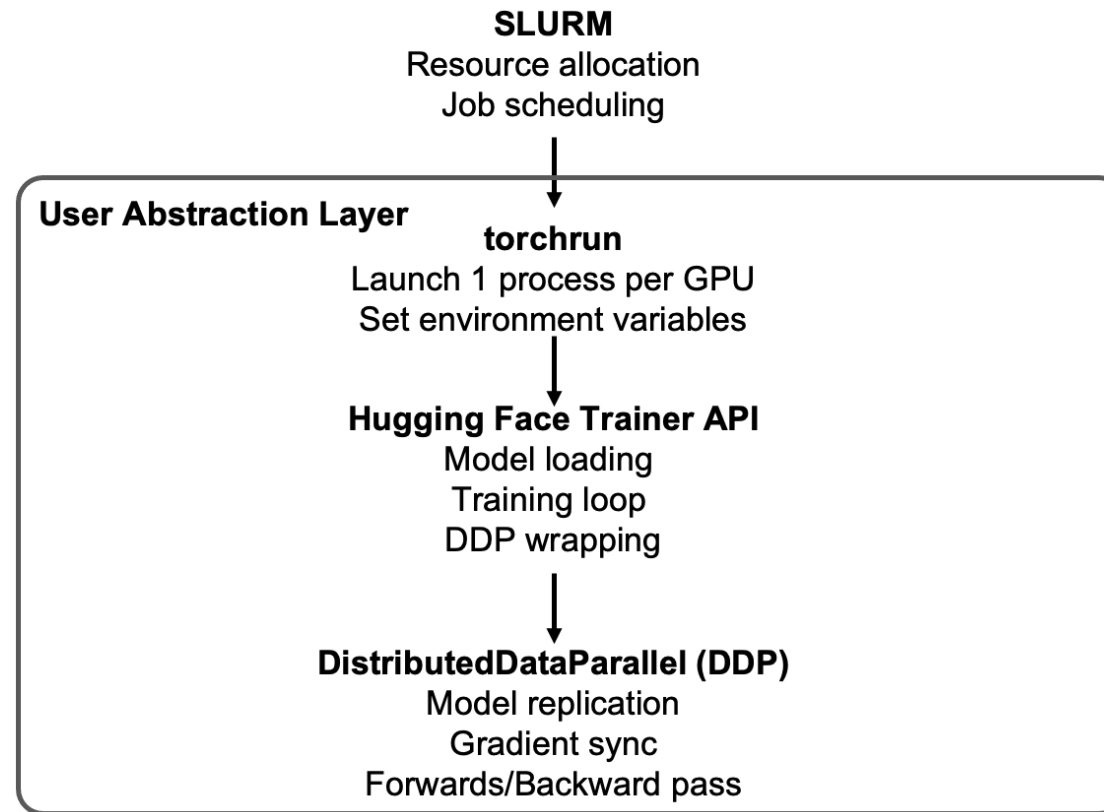


Case Study and Code Foundation

Case Study

■ Execution Flow:

- The training job is launched via torchrun in a SLURM-managed environment, which automatically initializes the Hugging Face Trainer API.



Baseline Training Script

- **benchmark.py**

- We establish a baseline using the Python script which trains a model on a single GPU using HF Trainer API.

```
import os
import torch

from transformers import AutoModelForCausalLM, Trainer
from utils import DummyDataset, get_args, log_rank, report_memory

def main(training_arguments, benchmark_arguments):
    # Set the data type for model parameters (bf16 or fp32)
    TORCH_DTYPE = torch.bfloat16 if benchmark_arguments.model_precision == "bf16"
    else torch.float

    world_size = int(os.environ["WORLD_SIZE"])

    model = AutoModelForCausalLM.from_pretrained(
        benchmark_arguments.path_to_model,
        attn_implementation=benchmark_arguments.attn,
        torch_dtype=TORCH_DTYPE
    )
```

Baseline Training Script

```
train_dataset = DummyDataset(  
    benchmark_arguments.sequence_length,  
    benchmark_arguments.num_samples  
)  
  
trainer = Trainer(  
    model=model,  
    args=training_arguments,  
    train_dataset=train_dataset  
)  
  
metrics = trainer.train().metrics  
log_rank(f"Training throughput:  {training_arguments.max_steps *  
    training_arguments.per_device_train_batch_size *  
    benchmark_arguments.sequence_length/metrics['train_runtime']:.2f}  
Tokens/s/GPU")  
  
report_memory()  
  
if __name__ == "__main__":  
    _training_arguments, _benchmark_arguments = get_args()  
    main(_training_arguments, _benchmark_arguments)
```

Visit section 13.3

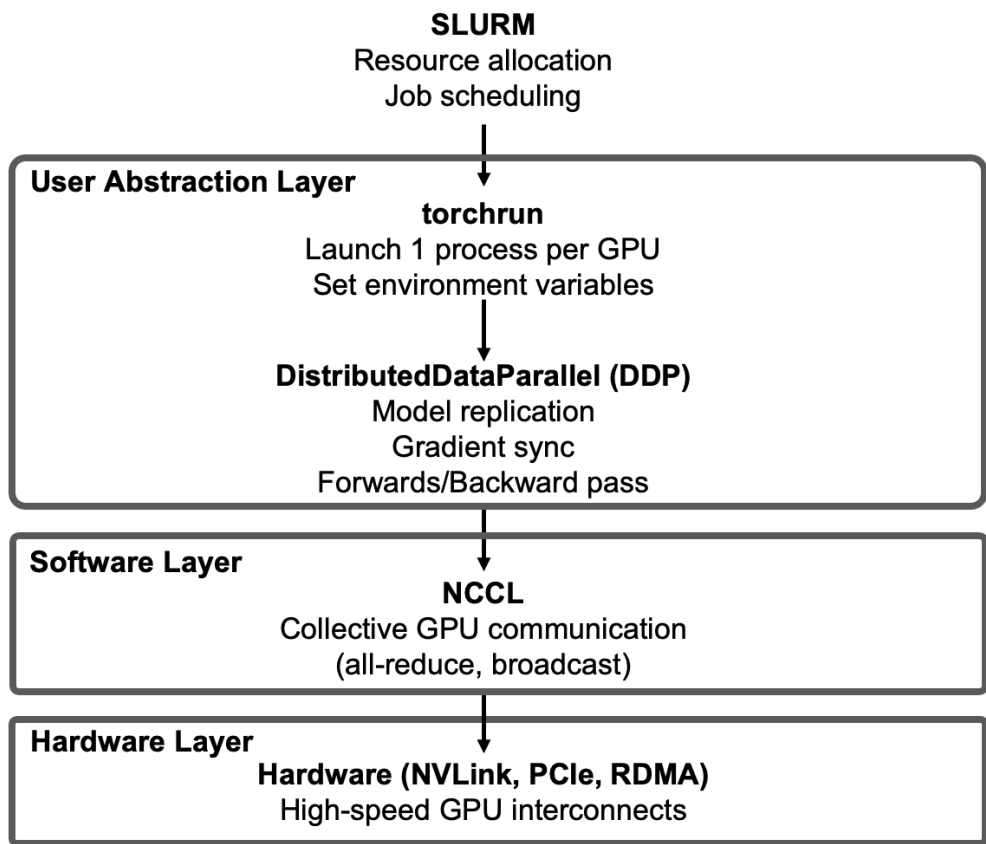
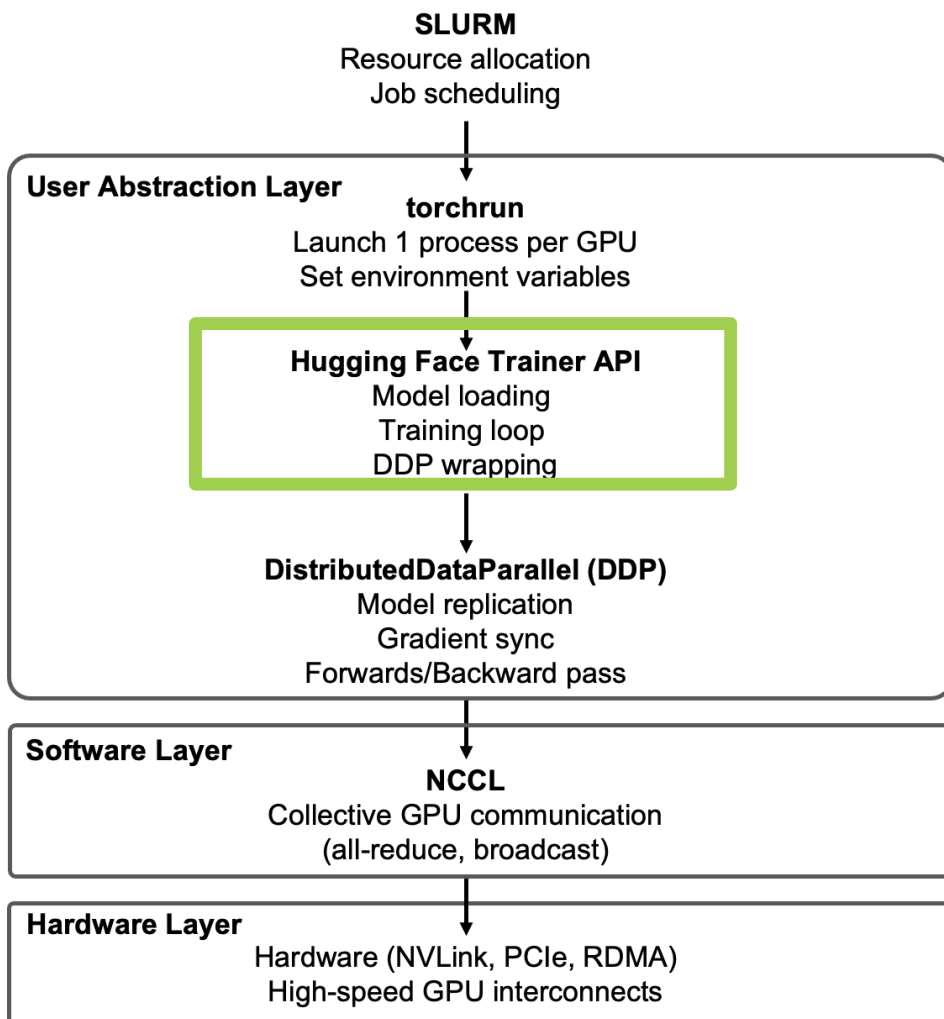
Chapter 13/14 slides:

■ The Role of Hugging Face in Modern LLMs

- Hugging Face has become the de facto standard platform for:
 - Sharing pre-trained models
 - Accessing large-scale datasets
 - Running reproducible AI experiments
- It provides:
 - A **model hub** with thousands of ready-to-use models
 - A **datasets library** for large-scale data access
 - High-level **training and inference APIs**

Chapter 13/14 slides:

- Hugging Face abstracts much of the engineering complexity of modern DL pipelines, especially for LLM.



Chapter 13/14 slides:

- **From Raw PyTorch to High-Level Training Abstractions**
 - In previous labs, we trained models using:
 - Explicit training loops
 - Manual optimization and logging
 - Direct control over GPUs and workers
 - Hugging Face introduces:
 - A **high-level training abstraction**
 - Automatic handling of: Optimization, Gradient accumulation, Mixed precision, etc.
- **We move from “how to train” to “what to train and how to configure it”.**

Chapter 13/14 slides:

- **Trainer is a high-level training engine (and API) built on top of PyTorch**
 - It manages automatically:
 - Training loop
 - Backpropagation
 - Evaluation
 - Logging
 - Checkpoint saving
 - The user only needs to provide:
 - A model
 - A dataset
 - A set of training arguments

Model + Dataset + TrainingArguments → Trainer → Full Training Pipeline

Chapter 13/14 slides:

■ Why Use Trainer in an HPC / LLM Context?

- Large models require:
 - Careful memory management
 - Efficient mixed precision
 - Scalable distributed training
- Trainer integrates natively with:
 - Data parallelism
 - Mixed precision
 - Gradient accumulation
 - Checkpoint resuming
 - ...

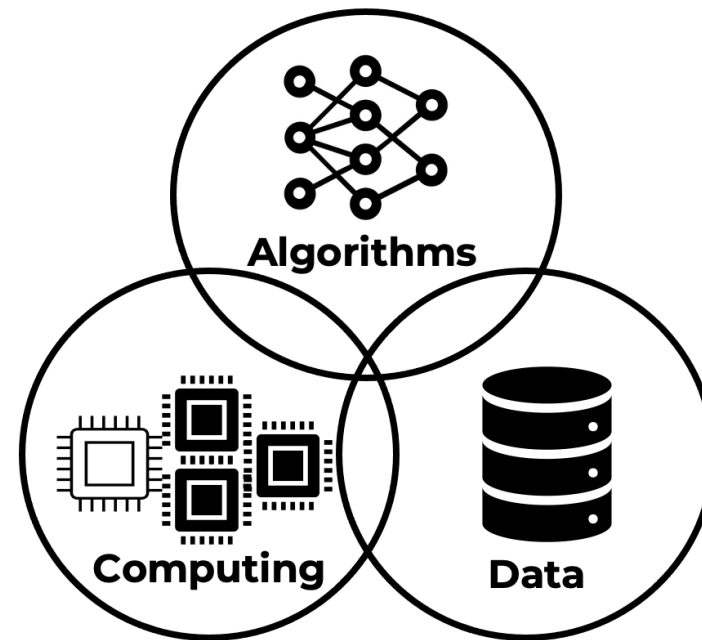
Chapter 13/14 slides:

- **This makes Trainer especially suited for:**
 - Large Language Models
 - Long-running supercomputing jobs
- **Key message:**
 - Trainer formalizes best practices that we previously implemented manually.

Custom PyTorch Loop	Hugging Face Trainer
Full control	High abstraction
More code	Minimal code
Manual logging	Automatic logging
Manual evaluation	Built-in evaluation
Manual checkpointing	Automatic checkpointing

LAB Pk: Optimization and Scaling of LLMs

Case Study in terms of



Case Study

■ Model:

- LLaMA 3.2 with 1 Billion parameters. (Used as a lightweight, representative LLM benchmark).

Location: `/gpfs/projects/<group_account>`

■ Data:

- Training uses a synthetic dataset (`DummyDataset`) to ensure controlled and efficient benchmarking.
 - Synthetic dataset of 10,000 samples generated dynamically.
 - This approach removes variability introduced by I/O operations or preprocessing, ensuring that performance metrics reflect the computational efficiency of the training process.
 - This approach allowing us to focus on GPU /Memory

```
from utils import DummyDataset, get_args, log_rank, report_memory
```

LAB Code?

/gpfs/projects/nct_345

```
[nct@alogin1]$ ls -ltr
```

```
drwxr-sr-x 2 nct00006 nct_345          4096 Jun 28 13:07 facebook-opt-1.3b
drwxr-sr-x 4 nct00006 nct_345          4096 Jun 28 13:07 Llama-3.2-1B
```

<https://github.com/jorditorresBCN/HPC4AIbook/Chapter.15>

Name	Last commit message
 ..	
 benchmark.py	Last update
 submit.LLM.template.slurm	update slurm template
 utils.py	Last update

Case Study Perf. Metrics

■ • Performance Metrics:

- Training Throughput: Measured in Tokens/s/GPU.

$$\text{Training Throughput} = \frac{\text{max_steps} \times \text{per_device_train_batch_size} \times \text{sequence_length}}{\text{train_runtime}}$$

- Max Reserved GPU Memory: Peak memory usage in GB
Obtained using `torch.cuda.max_memory_reserved()`, this value indicates peak memory usage during training.

Example:

```
INFO -> Training throughput:      27542.25 Tokens/s/GPU
INFO -> Max reserved GPU Memory: 61.59
```

SLURM Job Submission Script

- **Experiments focus on performance**

- We can run all experiments by editing only the SLURM script.
- Python code remains fixed.

- **Fixed variables to ensure consistency across experiments:**

- MAX_STEPS
 - Maximum number of training steps.
- SEQUENCE_LEN
 - Input sequence length in tokens. (Kept short to accommodate course resource limitations.)
- OPTIMIZER
 - Optimizer used; fixed to AdamW.
- TORCH_COMPILE
 - Whether to use `torch.compile`. Disabled due to current performance issues with Liger kernels.

SLURM Job Submission Script

- **The variables to be changed across experiments include:**
 - MICRO_BATCH_SIZE
 - Number of samples per device in each forward/backward pass.
 - MODEL_PRECISION
 - Precision format (fp32, bf16).
 - MIXED_PRECISION
 - Enables AMP (Automatic Mixed Precision).
 - ATTN
 - Attention implementation: eager, sdpa, flash_attention_2.
 - LIGER_KERNEL
 - Whether optimized kernels are used.

Running the Baseline Experiment

- **Default SLURM script for LLaMA 3.2 model**
 - Results are recorded in a Table 15.1 that tracks the value of each variable per run.

Task	Num GPUs	Batch Size	Model Precision	Mixed Precision	Attention Type	Liger	Training Throughput	Memory (GB)
15.1	1	6	fp32	No	eager	false	10485	61,89

Table 15.1 – Baseline training results using LLaMA 3.2 (1B) model.

Task 15.1

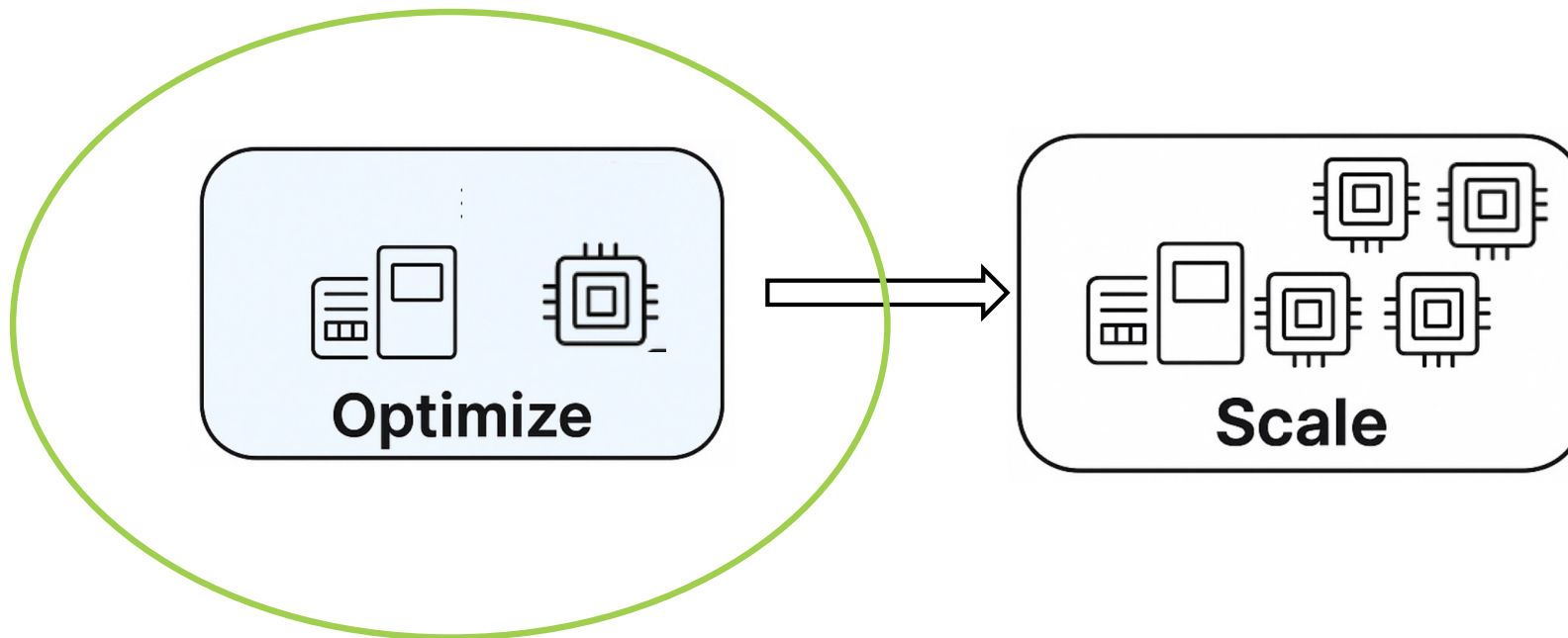


LAB Pk: Optimization and Scaling of LLMs

Optimize

Before scaling out

- Always optimize the single-GPU bottleneck first before distributing the workload to maximize overall system efficiency.



Efficient Training on a Single GPU

■ Optimization Focus:

- Tuning configurations that heavily influence throughput and memory utilization, including:

1. Batch Size Scaling.
2. Numerical Precision (Mixed Precision, Model Precision).
3. Attention Mechanism Efficiency.
4. Optimized Kernels (Liger).

Batch Size Scaling and Memory Limits

■ Strategy:

- The largest possible batch size that fits into GPU memory is sought to maximize throughput.

■ Constraint:

- Incrementally increasing the batch size is halted by the Out-of-Memory (OOM) limit of the GPU.

■ Baseline Status (FP32):

- The baseline experiment used a Batch Size of 6, while a Batch Size of 7 immediately resulted in an OOM error.

Task	Num GPUs	Batch Size	Model Precision	Mixed Precision	Attention Type	Liger	Training Throughput	Memory (GB)
15.1	1	6	fp32	No	eager	false	10,485	61,89
15.2	1	7	fp32	No	eager	false	-	OOM

Task 15.2

Enhancing Training Efficiency with Mixed Precision

■ Technique:

- Mixed Precision Training (Automatic Mixed Precision, or AMP).

■ Format Choice:

- Using bfloat16 (bf16) is preferred for LLMs due to its wide dynamic range (same 8-bit exponent as FP32), which aids numerical stability.

■ Hardware Advantage:

- bf16 utilizes the GPU's dedicated Tensor Cores, resulting in faster training and modest memory savings.

■ Implementation:

- Enable mixed precision by modifying the `MIXED_PRECISION` variable in the SLURM script.

Enhancing Training Efficiency with Mixed Precision

■ Results:

Task	Num GPUs	Batch Size	Model Precision	Mixed Precision	Attention Type	Liger	Training Throughput	Memory (GB)
15.1	1	6	fp32	No	eager	false	10,485	61,89
15.2	1	7	fp32	No	eager	false	-	OOM

Table 15.2 – Training throughput and memory usage using fp32 precision.

Task	Num GPUs	Batch Size	Model Precision	Mixed Precision	Attention Type	Liger	Training Throughput	Memory (GB)
15.3	1	5	fp32	bf16	eager	false	11.489	58,27

Table 15.3 – Training with mixed precision (bf16) reduces memory consumption and increases throughput.

Task 15.3

Fine-Tuning Model Precision

- **Technique:**

- Model Precision optimization involves storing all model weights, activations, and gradients entirely in bf16.

- **Benefit:**

- This approach eliminates the need for FP32 master copies, resulting in a significant reduction in memory usage (nearly 10 GB saved), which, in turn, allows for even larger batch sizes.

- **Implementation:**

- Set the `MODEL_PRECISION` variable to `bf16` in the SLURM script.

Fine-Tuning Model Precision

■ Results:

Task	Num GPUs	Batch Size	Model Precision	Mixed Precision	Attention Type	Liger	Training Throughput	Memory (GB)
15.3	1	5	fp32	bf16	eager	false	11.489	58,27

Table 15.3 – Training with mixed precision (bf16) reduces memory consumption and increases throughput.

Task	Num GPUs	Batch Size	Model Precision	Mixed Precision	Attention Type	Liger	Training Throughput	Memory (GB)
15.4	1	5	bf16	bf16	eager	false	13.520	47,66

Table 15.4 – Using bf16 model precision further improves throughput and reduces memory usage.

- The result is a significant drop in memory usage—nearly 10 GB saved—along with a noticeable increase in throughput. This optimization opens the door to larger batch sizes.

Task 15.4

Fine-Tuning Model Precision

■ Larger batch size:

- With memory freed by using bf16 for both model and mixed precision, we can now increase the batch size. Using the same configuration, we gradually scale up the batch size and find that the maximum batch size without an OOM error is now 7—up from 5.

Task	Num GPUs	Batch Size	Model Precision	Mixed Precision	Attention Type	Liger	Training Throughput	Memory (GB)
15.5	1	7	bf16	bf16	eager	false	14.315	61,32
15.5	1	8	bf16	bf16	eager	false	-	OOM

Table 15.5 – Model precision optimizations allow a larger batch size and improved throughput.

Task 15.5

Enhancing Attention Mechanisms

■ Technique

- Utilizing Flash Attention, an advanced implementation of the self-attention mechanism.

■ Benefit

- Flash Attention significantly reduces memory usage (as it is optimized for memory-bound operations) and boosts throughput.

■ Implementation

- Set `ATTN="sdpa"` in the configuration.

Enhancing Attention Mechanisms

■ Result

Task	Num GPUs	Batch Size	Model Precision	Mixed Precision	Attention Type	Liger	Training Throughput	Memory (GB)
6	1	7	bf16	bf16	sdpa	false	24.446	39,27

Table 15.6 – Flash Attention drastically reduces memory usage and boosts throughput.

- This optimization enables a further increase in batch size:

Task	Num GPUs	Batch Size	Model Precision	Mixed Precision	Attention Type	Liger	Training Throughput	Memory (GB)
15.7	1	14	bf16	bf16	sdpa	false	27.669	60,89

Table 15.7 – Flash Attention enables a significantly larger batch size and higher throughput.

Task 15.6

Task 15.7

Accelerating Training with Triton and Liger Kernels

■ Technique

- Enabling Liger Kernels, an open-source set of GPU kernels optimized using the Triton framework (developed by OpenAI).

■ Purpose:

- Liger optimizes memory-bound operations within the Transformer architecture, providing additional memory headroom.

■ How?

- Liger kernels are seamlessly integrated into Hugging Face Transformers, requires only a simple flag,
`--use_liger_kernel true`.
- Enabling them only requires setting `LIGER_KERNEL=true` in the SLURM script.

Accelerating Training with Triton and Liger Kernels

■ Result:

- By enabling Liger kernels, we observe a dramatic reduction in memory usage—nearly half—and a substantial increase in training throughput.

Task	Num GPUs	Batch Size	Model Precision	Mixed Precision	Attention Type	Liger	Training Throughput	Memory (GB)
15.8	1	14	bf16	bf16	sdpa	true	36.479	32,46

Table 15.8 – Liger kernels reduce memory usage by ~50% and improve throughput by 32%.

Accelerating Training with Triton and Liger Kernels

■ Result:

- This final optimization allows the batch size to increase to 37 (over $6\times$ the original baseline) and achieves a single-GPU throughput of over 48,000 tokens/s.

Task	Num GPUs	Batch Size	Model Precision	Mixed Precision	Attention Type	Liger	Training Throughput	Memory (GB)
15.9	1	37	bf16	bf16	sdpa	true	48.081	62,33

Table 15.9 – Final optimized configuration achieves $4.5\times$ throughput improvement over the baseline.

Task	Num GPUs	Batch Size	Model Precision	Mixed Precision	Attention Type	Liger	Training Throughput	Memory (GB)
15.1	1	6	fp32	No	eager	false	10485	61,89

Table 15.1 – Baseline training results using LLaMA 3.2 (1B) model.

Task 15.8

Task 15.9

Single-GPU Optimization Summary

■ Total Gain

- The systematic application of optimization techniques resulted in a $4.5\times$ increase in training throughput on a single H100 GPU.

Task	Num GPUs	Batch Size	Model Precision	Mixed Precision	Attention Type	Liger	Training Throughput	Memory (GB)
15.1	1	6	fp32	No	eager	false	10485	61,89

Table 15.1 – Baseline training results using LLaMA 3.2 (1B) model.

Task	Num GPUs	Batch Size	Model Precision	Mixed Precision	Attention Type	Liger	Training Throughput	Memory (GB)
15.9	1	37	bf16	bf16	sdpa	true	48.081	62,33

Table 15.9 – Final optimized configuration achieves $4.5\times$ throughput improvement over the baseline.

■ Takeaway:

- This demonstrated how performance can be extracted from high-end GPU architectures (H100) through low-level, hardware-aware optimizations without modifying the model code.



LAB Pk: Optimization and Scaling of LLMs

Distributed Training Across Multiple GPUs

Distributed Training LLMs

- **requires not only powerful hardware but also sophisticated software frameworks capable of orchestrating computation across multiple GPUs and nodes.**
 - While frameworks such as DeepSpeed, Megatron-LM, and NVIDIA NeMo provide advanced tools for distributed model and data parallelism, their complexity falls beyond the scope of this book.
- **Our goal**
 - demonstrate the performance impact of distributed training using **Hugging Face's Trainer API**.

Distributed Training LLMs

■ Experiment

- Performance is measured across 2 to 32 GPUs on the MareNostrum 5 supercomputer.

■ Experiment Setup

- Baseline Configuration:
 - The single-GPU optimized setup (Task 15.9: bf16, Flash Attention, Liger, Batch Size 37 per GPU).
- Scaling Framework:
 - Hugging Face Trainer API is utilized,
 - delegating model replication and gradient synchronization to PyTorch DDP (Distributed Data Parallel).

■ Goal

- Quantify the speedup and parallel efficiency when scaling LLM training across multiple nodes.

Results

Num GPUs	Batch Size	Model Precision	Mixed Precision	Attention Type	Liger	Training Throughput	Memory (GB)
1	37	bf16	bf16	sdpa	true	48,229	62.34
2	37	bf16	bf16	sdpa	true	46,188	62.21
4	37	bf16	bf16	sdpa	true	43,432	61.97
8	37	bf16	bf16	sdpa	true	42,297	62.13
16	37	bf16	bf16	sdpa	true	41,869	61.99
32	37	bf16	bf16	sdpa	true	41,424	61.99

Table 15.10 – Per-GPU training throughput and memory usage when scaling training of the model across multiple GPUs on MareNostrum 5.

Scaling Results and Efficiency

Num GPUs	Per-GPU Throughput	Total Throughput	Speedup vs 1 GPU	Efficiency (%)
1	48,229	48,229	1.00×	100%
2	46,188	92,376	1.92×	96%
4	43,432	173,728	3.60×	90%
8	42,297	338,376	7.01×	88%
16	41,869	669,904	13.89×	87%
32	41,424	1,325,568	27.48×	86%

Table 15.11 – Total training throughput, relative speedup, and parallel efficiency of distributed training using up to 32 GPUs. Efficiency is computed as Speedup divided by the number of GPUs.

Scaling Results and Efficiency

■ Total Throughput

- scaled significantly, achieving a speedup of $27.48 \times$ at 32 GPUs compared to the single-GPU baseline.

■ Parallel Efficiency

- Efficiency remained remarkably high, decreasing gradually from 100% (1 GPU) to 86% (32 GPUs).

■ Observation:

- The slight decline in efficiency is attributed to the increasing cost of inter-GPU communication and system overheads.

Task 15.10

Tokens Processed Per Euro Spent

- If GPU usage is billed linearly over time (as in cloud platforms or shared HPC environments), then:
 - $\text{Cost} \propto \text{Training Time}$
 - $\text{Training Time} \propto \text{Tokens to Process} / \text{Total Throughput}$
- Thus, to reduce costs,
 - we must maximize throughput **per GPU-hour**.
 - However, the results show that while total throughput increases with more GPUs, the per-GPU efficiency decreases. This introduces a trade-off: training may finish sooner, but the cost per unit of progress may increase.

Cost-Efficiency Analysis

Num GPU_s	Total Throughput	Efficiency (Tokens/s/GPU)	Cost Efficiency (Tokens/s/€)
1	48,229	100%	1.00×
2	92,376	96%	0.96×
4	173,728	90%	0.90×
8	338,376	88%	0.88×
16	669,904	87%	0.87×
32	1,325,568	86%	0.86×

Table 15.12 – Cost-efficiency analysis based on throughput per GPU and normalized economic cost per token processed. Assumes linear billing proportional to GPU time.

Cost-Efficiency Analysis

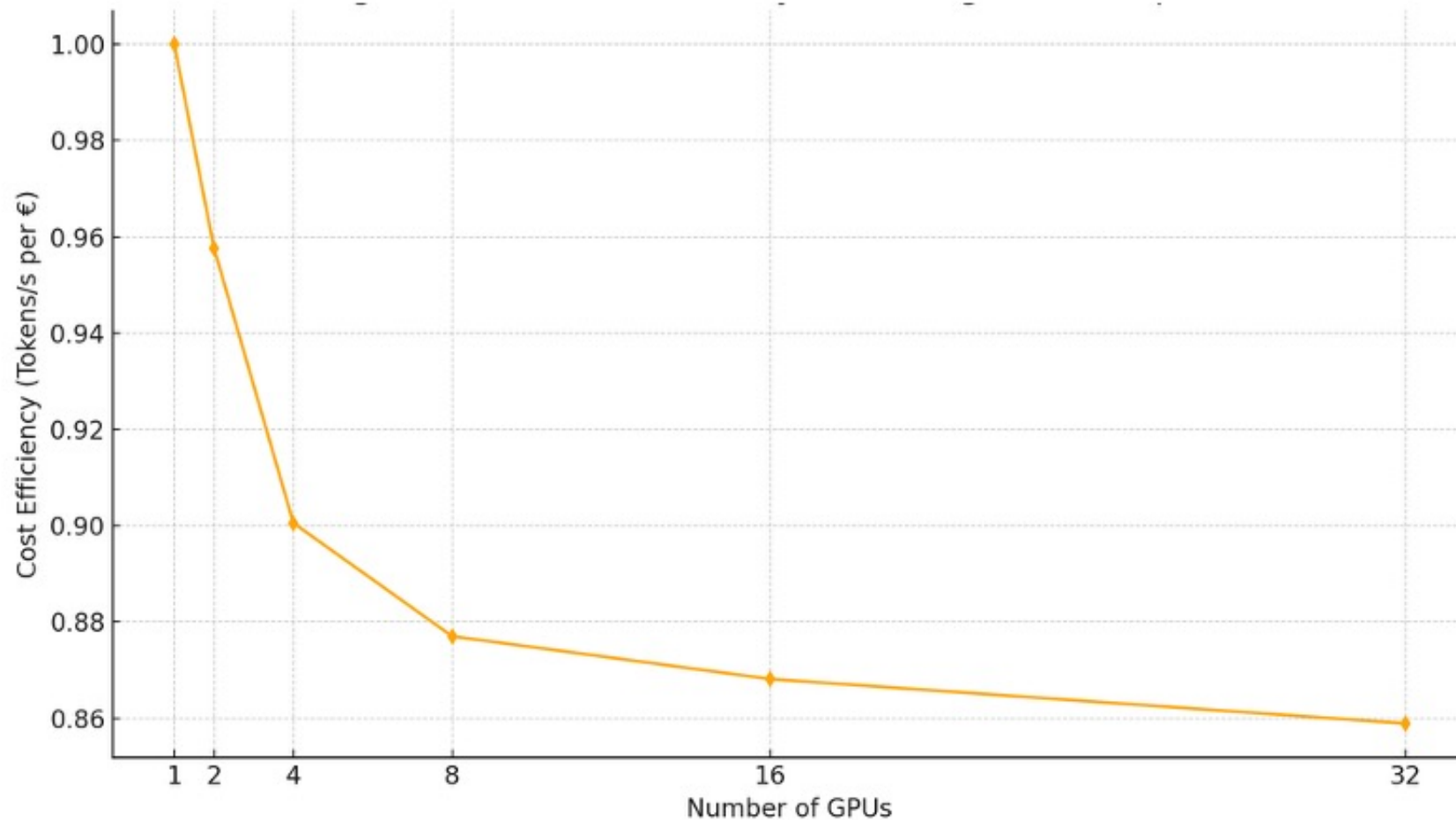


Figure 15.1 – Cost-efficiency of training (proxy metric: tokens/s/€). Although total throughput scales, the efficiency per euro slightly declines with more GPUs.

Cost-Efficient Optimization: Performance per Euro

■ Concept:

- Optimization must balance raw performance with cost-efficiency → e.g., tokens processed per second per euro spent.

■ Trade-off:

- While 32 GPUs offer the maximum speed, cost-efficiency slightly declines as scale increases (86% efficiency at 32 GPUs vs. 100% at 1 GPU).

■ Insight:

- Adding GPUs provides diminishing returns in efficiency beyond a certain scale. Researchers must identify the "sweet spot" based on time constraints versus budget.

Task 15.11

Task 15.12

Task 15.13

Task 15.14

Key Takeaways from Chapter 15

- **Single-GPU Optimization:**

- Fine-grained optimizations (e.g., precision tuning, kernel fusion) are highly effective, yielding a $4.5\times$ throughput increase on a single GPU.

- **Distributed Scaling:**

- The optimized configuration scaled with excellent efficiency (86% at 32 GPUs), demonstrating robust performance for large models using DDP/NCCL on supercomputers.

- **Workflow Simplicity:**

- The Hugging Face Trainer API allowed deployment across 1 to 32 GPUs with minimal code changes, effectively managing the underlying distributed stack.

Pk: Optimization and Scaling of LLMs

- **Tasks included:**

- task 15.1- task 15-11

- **Deliverable:**

- Upload a single PDF (per group) to the racó@FIB intranet, containing as many slides as you need per task to clearly express what is requested in each one.

Lab Pg presentation

- 1 group **will be randomly chosen**
 - We'll sum 4 numbers from randomly chosen students and use the '%' function with the total number of students to find the winner in the list.

```
>>> nums_to_add = ...+...+...+...
>>> winner= nums_to_add %
num_students +1
>>> print (winner)
```

1	ALONSO GARCIA, MARTÍ;
2	ANDREU GERIQUE, DAVID;
3	ARNABOLDI, LEONARDO;
4	ARÓSTEGUI GARCÍA, ALBERTO;
5	AUBERTIER, THOMAS;
6	CRAMER, TIM;
7	DAHLBOM, IRMA;
8	DÍAZ CALDERÓN, MARC;
9	ESCOLÀ CURCÓ, POL;
10	HENLE, PATRICK;
11	HÖPFL, LEON;
12	KARG, CHISTIAN;
13	KOCH ENES, ANTONIO MANUEL;
14	KOLEVA, ELIZABET;
15	LÓPEZ CASTILLÓN, PAU;
16	MENADOZ, VÍCTOR;
17	MOLDOVEANU, ANDREI;
18	OLIVERA LOYOLA, JUAN JOSE;
19	PAXTON, LAURA;
20	RABADAN ARROYO, RAÛL;
21	RIOS LÓPEZ, ADRIÁN;
22	RITTER, MAXIMILIAN FALK;
23	SELIGA, JAKUB;
24	SIMON, WILLIAM;
25	SOLDERA GIRELLI, VALÉRIA;
26	WALSH, ERIC;